GSHTrans: mathematics and numerical methods

A reference for what the library does, and why

Abstract

This is a record rather than a plan. Part I collects the definitions, conventions and relations the library rests on, in the form it actually uses them, with a translation table for the three sources whose notations differ. Part II explains the numerical methods: what the algorithms are, why they were chosen, and the handful of places where the reasoning is subtle enough to be worth writing down. Neither part is a derivation. The companion note `canonical-components.tex` carries the mathematical arguments; `core-plan.md` and `field-algebra-plan.md` carry the design decisions and the measurements behind them.

Statements here are either verified by a test in the library or read directly from a cited source. The two places where a convention remains genuinely open are marked as such.

Contents

Part I
Mathematics

1 Conventions

1.1 The canonical basis

At each point of the sphere, with $\hat{\mathbf{r}}, \hat{\boldsymbol{\theta}}, \hat{\boldsymbol{\phi}}$ the usual orthonormal spherical polar frame,

$$\mathbf{e}_{\pm} = \mp\tfrac{1}{\sqrt{2}}\left(\hat{\boldsymbol{\theta}} \pm i\hat{\boldsymbol{\phi}}\right), \qquad \mathbf{e}_0 = \hat{\mathbf{r}}, \qquad \alpha \in \{-1, 0, +1\}. \tag{1}$$

Two consequences are used everywhere:

$$\overline{\mathbf{e}_\alpha} = (-1)^\alpha \mathbf{e}_{-\alpha}, \qquad g_{\alpha\beta} = \mathbf{e}_\alpha \cdot \mathbf{e}_\beta = (-1)^\alpha \delta_{\alpha+\beta,0}. \tag{2}$$

So $g_{+-} = g_{-+} = -1$, $g_{00} = 1$, and g is its own inverse. The library works with contravariant components throughout and applies g explicitly wherever a contraction is taken; index position is not tracked.

Under a rotation of the tangent frame through ψ about $\hat{\mathbf{r}}$, $\mathbf{e}_\pm \mapsto e^{\mp i\psi}\mathbf{e}_\pm$.

1.2 What the library stores, exactly

The single most useful statement for reading the code:

> `Wigner`'s stored value at upper index N, degree l and order m is
>
> $$\left(\frac{2l+1}{4\pi}\right)^{1/2} d^l_{Nm}(\theta) \;=\; X^N_{lm}(\theta), \tag{3}$$
>
> which is exactly the scalar harmonic X^N_{lm} of Dahlen & Tromp (1998), their (C.117), built on their generalized Legendre function $P^N_{lm} = d^l_{Nm}$ — *upper index first.*

This is verified rather than assumed. `tests/CheckWignerConvention.h` compares all nine $l = 1$ values against D&T (C.115); the check discriminates, because the four entries with $N - m$ odd distinguish d^l_{Nm} from d^l_{mN}, which differ by $(-1)^{N-m}$. Two further facts corroborate it: the recursion seed evaluates to D&T (C.125), and `tests/CheckLegendre.h` pins the $N = 0$ row against `std::sph_legendre`, fixing the orthonormal scaling and the Condon–Shortley phase.

1.3 Translation table

The three standard sources use different notations, and two of them differ from this library in normalisation. Everything in this table is read directly from the cited equations.

	harmonic	colatitudinal part	normalised?
GSHTrans	$Y^N_{lm} = X^N_{lm} e^{im\phi}$	$X^N_{lm} = \left(\frac{2l+1}{4\pi}\right)^{1/2} d^l_{Nm}$	yes
Dahlen & Tromp (C.117)	$Y^N_{lm} = X^N_{lm} e^{im\phi}$	$X^N_{lm} = \left(\frac{2l+1}{4\pi}\right)^{1/2} P^N_{lm}$	yes
Woodhouse & Deuss [41]	$Y^{Nm}_l = P^{Nm}_l e^{im\phi}$	$P^{Nm}_l = d^{(l)}_{Nm}$	**no**
Phinney & Burridge (1.5)	$X^{Nm} = P^{Nm}_l e^{im\phi}$	P^{Nm}_l, their Appendix A	**no**

Three things to take from it.

The index order is the same in all four. D&T write P^N_{lm}, Woodhouse and P&B write P^{Nm}_l, and this library writes d^l_{Nm}; all mean the same function with the upper index first. D&T state that they follow P&B in the definition.

The normalisation is not. D&T and this library carry the $[(2l+1)/4\pi]^{1/2}$; Woodhouse and P&B do not. A formula taken from Woodhouse or P&B therefore needs that factor supplied, and a formula taken from D&T does not.

The transposed function is a different thing. D&T (C.118) gives $P^m_{lN} = (-1)^{m+N} P^N_{lm}$, so getting the index order wrong costs a sign whenever $m - N$ is odd, and nothing when it is even — which is exactly the kind of error that survives half a test suite.

1.4 What remains a convention

This document used to list two statements as unsettled: the sign $\mp$ in (1), since some authors use $\mathbf{e}_\pm = \pm(\hat{\boldsymbol{\theta}} \pm i\hat{\boldsymbol{\phi}})/\sqrt{2}$, which flips every odd-rank reality phase; and the overall signs of $\eth$ and $\overline{\eth}$ (§6). Each was said to be unobservable, and each was, for the same reason: everything the library computed stayed in canonical components from end to end, and a convention that is never read back into the physical frame cannot be seen.

They are now pinned, jointly, by reading a gradient back into the physical frame and requiring it to be the gradient. Inverting (1) for a vector $\mathbf{v} = v^\alpha \mathbf{e}_\alpha$,

$$v_\theta = \frac{v^- - v^+}{\sqrt{2}}, \qquad v_\phi = -i\,\frac{v^+ + v^-}{\sqrt{2}}, \qquad v_r = v^0, \tag{4}$$

and `tests/TestConventions.cpp` requires that for a scalar f the surface gradient satisfies $\mathbf{v} = \hat{\boldsymbol{\theta}}\,\partial_\theta f + \hat{\boldsymbol{\phi}}\,(\sin\theta)^{-1}\partial_\phi f$ pointwise, and that the metric trace of the surface gradient of a tangential vector field is its surface divergence. Both hold to 10^{-13}, and both fail by an $O(1)$ amount if either sign is flipped alone.

What is left is genuinely a convention and nothing more: flipping (1) *and* the sign of the $\eth$ pair together is a relabelling of which component is called $+1$, and (4) flips with it. So there is one convention here rather than two unknowns, this document states it, and a test observes it.

A trap worth recording. A tangential field written as $a(\theta,\phi)\,\hat{\boldsymbol{\theta}}$ is almost never smooth, because $\hat{\boldsymbol{\theta}}$ is not: at a pole its direction depends on the azimuth from which the pole is approached. The innocuous-looking $a = \sin\theta\cos\phi$ is not differentiable there, its spin-weighted expansion does not converge, and the divergence check above came out wrong by 0.18 at every truncation from $l_{\max} = 8$ to 128. That is what a non-convergent expansion looks like, and it is easily mistaken for a bug in the operator. The test fields are built as $\nabla_1 f + \hat{\mathbf{r}} \times \nabla_1 g$ from smooth scalars, which is smooth by construction.

2 Canonical components

A rank-p tensor field has 3^p scalar components, each labelled by a multi-index $(\alpha_1 \dots \alpha_p) \in \{-1, 0, 1\}^p$. Under the frame rotation, the component $T^{\alpha_1 \cdots \alpha_p}$ acquires the phase $e^{-iN\psi}$ with

$$N = \sum_{i=1}^{p} \alpha_i, \tag{5}$$

and it is this N — not the multi-index — that is the upper index of the component's harmonic expansion.

The multi-index is not the upper index. For a vector they coincide, $N = \alpha$, which is the coincidence that misleads. For $p \geq 2$ distinct components share an upper index: a rank-2 tensor has three at $N = 0$, namely $(-+)$, (00) and $(+-)$. A collection of components labelled only by N therefore does not determine a tensor. The number carrying a given N is the trinomial coefficient; for $p = 2$ these are $1, 2, 3, 2, 1$ across $N = -2 \dots 2$.

Products and contractions. Concatenating multi-indices adds upper indices by (5), which is the statement "upper indices add under pointwise multiplication". Contracting slots j and k against the metric gives

$$(\mathrm{tr}_{jk} T)^{\cdots} = \sum_{\alpha} (-1)^\alpha\, T^{\cdots\,\alpha\,\cdots\,-\alpha\,\cdots} = -T^{\cdots - \cdots + \cdots} + T^{\cdots 0 \cdots 0 \cdots} - T^{\cdots + \cdots - \cdots}, \tag{6}$$

the contracted pair contributing $\alpha + (-\alpha) = 0$, so the result carries the upper index of the surviving slots.

3 Generalized spherical harmonics

A field f of upper index N expands as

$$f(\theta,\phi)=\sum_{l\geq|N|}\sum_{m=-l}^{l} f^N_{lm}Y^N_{lm},\qquad f^N_{lm}=\int_{S^2}\overline{Y^N_{lm}}\,f\,\mathrm{d}\Omega, \tag{7}$$

with

$$Y^N_{lm}(\theta,\phi)=\left(\frac{2l+1}{4\pi}\right)^{1/2} d^l_{Nm}(\theta)\,e^{im\phi}. \tag{8}$$

These are orthonormal, $\int\overline{Y^N_{lm}}Y^N_{l'm'}\,\mathrm{d}\Omega=\delta_{ll'}\delta_{mm'}$. The sum starts at $l=|N|$ because d^l_{Nm} vanishes identically below it: there is no harmonic of that upper index at lower degree, and correspondingly no coefficient to store. This is the single fact most likely to look like a bug on first contact — see §9.3.

The normalisation is pinned by the code. Taking f constant, (7) gives $f^0_{00}=2\sqrt{\pi}f$, and `GaussLegendreGrid` implements exactly that on its one-point grid.

Conjugation. Using $d^l_{-m,-N}=(-1)^{m-N}d^l_{mN}$ and the reality of d, equation (8) gives D&T (C.109),

$$\overline{Y^N_{lm}}=(-1)^{m+N}Y^{-N}_{l,-m}, \tag{9}$$

from which follow the two spectral reality relations of §5.

4 The generalized Legendre functions

$P^N_{lm}(\mu)=d^l_{Nm}(\theta)$, $\mu=\cos\theta$, is the regular solution of the generalized Legendre equation, normalised as D&T (C.112) so that the addition theorem below holds. It does *not* reduce to the associated Legendre function at $N=0$; D&T (C.113) relates them.

4.1 Symmetries

D&T (C.118), in full, since the library uses two of them and could use two more:

$$\begin{aligned}
P^N_{l,-m}(\mu)&=(-1)^{l+N}P^N_{lm}(-\mu), & P^{-N}_{lm}(\mu)&=(-1)^{l+m}P^N_{lm}(-\mu),\\
P^{-N}_{l,-m}(\mu)&=(-1)^{m+N}P^N_{lm}(\mu), & P^m_{lN}(\mu)&=(-1)^{m+N}P^N_{lm}(\mu),\\
P^{-m}_{l,-N}(\mu)&=P^N_{lm}(\mu).
\end{aligned} \tag{10}$$

The third is the $(m,N)\mapsto(-m,-N)$ involution; the second is the $\theta\mapsto\pi-\theta$ reflection, which maps the Gauss–Legendre node set to itself exactly, since those nodes are symmetric about $\pi/2$. Composed, the two generate a group of order four acting on (N,m,θ), so the table of stored values could in principle be reduced fourfold. Neither reduction is implemented; §16 says why. The fourth relation is the transposition that the convention check discriminates against.

4.2 Boundary values

The values at the extreme orders have closed forms, D&T (C.123) and (C.125), and these are precisely the library's `WignerMinOrder` and `WignerMaxOrder`:

$$P^N_{l,-l}=\left[\frac{(2l)!}{(l+N)!(l-N)!}\right]^{1/2}\left(\sin\tfrac{\theta}{2}\right)^{l+N}\left(\cos\tfrac{\theta}{2}\right)^{l-N}, \tag{11}$$

$$P^N_{ll}=(-1)^{l+N}P^N_{l,-l}\big|_{N\to-N}. \tag{12}$$

They are the seeds of D&T's recursion in m, and the boundary terms of the library's recursion in l (§10).

4.3 The addition theorem

D&T (C.127),

$$\sum_{m=-l}^{l} P_{lm}^{N}(\mu)\, P_{lm}^{N'}(\mu) = \delta_{NN'}. \tag{13}$$

This is the strongest available check on a computed table, and `tests/CheckAdditionTheorem.h` is it. Because the library stores X_{lm}^{N} rather than P_{lm}^{N}, the right-hand side there is $\delta_{NN'}(2l+1)/4\pi$.

5 Reality

For a real tensor field, applying (2) to each factor gives

$$T^{-\alpha_1 \ldots -\alpha_p} = (-1)^N\, \overline{T^{\alpha_1 \ldots \alpha_p}}. \tag{14}$$

For a vector this reads $u^0 = \overline{u^0}$ and $u^- = -\overline{u^+}$: the radial component is a real field and the two transverse components are not independent.

Which components must be stored. (14) pairs each multi-index with its negation, and one representative of each orbit suffices. The only fixed point is the all-zero index, which is therefore a real-valued field; every other orbit has size two. With a permutation symmetry present the orbits are those of the group generated by the permutations *together with* the negation, and a component whose orbit contains its own negation is pinned to a single real number — real, or purely imaginary when a permutation sign intervenes. Such self-paired components always sit at $N = 0$: permutation preserves the slot sum and negation reverses it, so a component fixed by the combination satisfies $N = -N$.

The resulting counts, computed by the library rather than tabulated by hand, are the independent real degrees of freedom in every case:

	complex components	pinned	reals per point
rank 1	1	1	3
rank 2	4	1	9
rank 2, symmetric	2	2	6
rank 2, antisymmetric	1	1	3
rank 3, symmetric	4	2	10
rank 4	40	1	81
rank 4, elastic	8	5	21

The last row is worth noting: nothing in the machinery knows about elasticity, and $c_{ijkl} = c_{jikl} = c_{ijlk} = c_{klij}$ comes out at the familiar 21.

In the spectral domain the same relation takes two distinct forms, and it is worth keeping them apart. For *any* field f of upper index N, (9) gives the coefficients of $\overline{f}$, which has upper index $-N$:

$$\big(\overline{f}\big)_{l,-m}^{-N} = (-1)^{m-N}\overline{f_{lm}^{N}}. \tag{15}$$

For a negation-paired component of a *real tensor*, (14) supplies a further $(-1)^N$ and the two combine to

$$T_{l,-m}^{-N} = (-1)^m \overline{T_{lm}^{N}}, \tag{16}$$

which at $N = 0$ is the familiar condition on a real scalar field, and is what the reduced $m \geq 0$ coefficient storage implements.

6 Raising and lowering

$\eth$ and $\overline{\eth}$ change the upper index by one. On a basis function,

$$\eth\, Y_{lm}^{N} = -\sqrt{(l-N)(l+N+1)}\; Y_{lm}^{N+1}, \qquad \overline{\eth}\, Y_{lm}^{N} = +\sqrt{(l+N)(l-N+1)}\; Y_{lm}^{N-1}, \tag{17}$$

so in the spectral domain each is a multiplication by an l-dependent factor, with no coupling between different (l, m). They are the operations that connect different upper indices, and they are diagonal in (l, m) rather than local in (θ, ϕ) — which is why the library has two representations rather than one, and why an expression such as "the gradient of a product" is necessarily evaluated in both.

The surface gradient of a scalar has canonical components proportional to $\eth f$ and $\overline{\eth} f$, at upper index $+1$ and -1, with vanishing radial component.

Two identities, and a third that pins the rest. Composing the factors,

$$\overline{\eth}\eth\big|_{N=0} = -l(l+1), \qquad [\eth, \overline{\eth}] = -2N. \tag{18}$$

The first says $\overline{\eth}\eth$ is the surface Laplacian on a scalar; it pins the relative sign of the two operators and both magnitudes. The second is structural. Both are tested, and both survive flipping the sign of the pair — which is why this document long carried the overall sign as an open question for Phinney & Burridge.

It does not need one. P&B do not use $\eth$ as such, and the sign is not theirs to settle; what has to be right is that gradients of functions come out as gradients, which is checkable here. Requiring exactly that, in the physical frame, fixes the overall sign in (17) against the sign in (1): see §1.4. The two are one convention, stated in this document and observed by `tests/TestConventions.cpp`.

The degree ranges look after themselves. Raising a field at $N \geq 0$ produces one starting at $l = N + 1$, and the factor at $l = N$ is $\sqrt{0} = 0$: the coefficient with nowhere to go was going to vanish. Lowering produces a field starting at a lower degree, whose extra coefficient has no source and is zero, which is what having no content there means.

7 The contravariant derivative, and how it differs

Phinney & Burridge, and Dahlen & Tromp following them, work not with $\eth$ but with a *contravariant derivative* ∂^{σ}, $\sigma \in \{-, 0, +\}$, which takes the expansion coefficients of a rank-q tensor to those of the rank-$(q+1)$ tensor $\nabla\mathbf{T}$. It is closely related to $\eth$ and it is not the same operator, and the difference matters for anyone using this library on tensors.

7.1 The definition

With D&T (C.146),

$$\Omega_{l}^{\pm N} = \sqrt{\tfrac{1}{2}(l \pm N)(l \mp N + 1)}, \tag{19}$$

their (C.151)–(C.153) read

$$\partial^- T^{\alpha_1\ldots\alpha_q}_{lm} = r^{-1}\Big[\Omega_l^{+N}\, T^{\alpha_1\alpha_2\ldots\alpha_q}_{lm} - T^{(\alpha_1-1)\alpha_2\ldots\alpha_q}_{lm} - \cdots - T^{\alpha_1\alpha_2\ldots(\alpha_q-1)}_{lm}\Big], \tag{20}$$

$$\partial^0 T^{\alpha_1\ldots\alpha_q}_{lm} = \frac{d\,T^{\alpha_1\ldots\alpha_q}_{lm}}{dr}, \tag{21}$$

$$\partial^+ T^{\alpha_1\ldots\alpha_q}_{lm} = r^{-1}\Big[\Omega_l^{-N}\, T^{\alpha_1\alpha_2\ldots\alpha_q}_{lm} - T^{(\alpha_1+1)\alpha_2\ldots\alpha_q}_{lm} - \cdots - T^{\alpha_1\alpha_2\ldots(\alpha_q+1)}_{lm}\Big], \tag{22}$$

with the stipulation that any coefficient whose shifted index leaves $\{-1, 0, 1\}$ is zero. The components of the gradient are $(\nabla\mathbf{T})^{\sigma\alpha_1\ldots\alpha_q} = \partial^\sigma T^{\alpha_1\ldots\alpha_q}$: the operator *prepends a slot*, and the result carries upper index $\sigma + N$, which is (5) applied to the lengthened multi-index.

7.2 Two differences, one benign and one not

A factor of $\sqrt{2}$, which is only the basis normalisation. Comparing (19) with (17),

$$|\text{raising factor}| = \sqrt{(l-N)(l+N+1)} = \sqrt{2}\,\Omega_l^{-N}, \qquad |\text{lowering factor}| = \sqrt{2}\,\Omega_l^{+N}. \tag{23}$$

The $\sqrt{2}$ is exactly the $1/\sqrt{2}$ in $\mathbf{e}_\pm = \mp(\hat{\boldsymbol{\theta}} \pm i\hat{\boldsymbol{\phi}})/\sqrt{2}$, which appears explicitly in D&T's surface gradient (C.144). So on the unit sphere, up to that factor and to the sign convention of §6, ∂^+ is $\eth$ and ∂^- is $\overline{\eth}$ — *for a scalar.*

Connection terms, which are not benign. For $q \geq 1$ the bracket in (20) and (22) is not a multiplication. It subtracts the components with one slot index shifted, once per slot. Those terms are the connection: the canonical basis vectors themselves vary over the sphere,

$$\big[\pm\partial_\theta + i(\sin\theta)^{-1}\partial_\phi\big]\,\mathbf{e}_\alpha = \alpha\cot\theta\,\mathbf{e}_\alpha - \sqrt{2}\,\mathbf{e}_{\alpha\pm1}, \tag{24}$$

so differentiating a tensor field differentiates its basis too. A scalar has no slots and therefore no connection terms, which is why the two operators coincide there and only there.

7.3 The consequence for this library

> **Applying $\eth$ to each component of a tensor does not give the gradient of the tensor.** Use `SurfaceGradient`. It gives the $\eth$ of each component as a spin-weighted function, which for $q \geq 1$ is not a component of $\nabla\mathbf{T}$ and in general is not a component of anything.

This is a trap rather than a limitation: `Raise` and `Lower` are correct for what they claim — a single spin-weighted function — and are correct for the surface gradient of a scalar, which is the case most people reach for first. They are simply not the *ambient* tensor operator, and nothing in their types says so, because a tensor expansion's components are ordinary spin expansions.

7.4 The intrinsic derivative, where $\eth$ is exactly right

The paragraph above must not be read as forbidding component-wise $\eth$ altogether, because there is a case in which it is precisely the correct operator.

There are two derivatives on the sphere. ∂^σ is the *ambient* one: it differentiates the tensor and its basis, and the basis leaves the tangent plane. The *intrinsic* covariant derivative — the Levi-Civita connection of the induced metric, which is what surface differential geometry means by differentiation — is closed on *tangential* tensors, those whose every slot is ±1 with no radial index.

On such a tensor the connection terms vanish identically. Every shift $\alpha_i + \sigma$ with $\alpha_i, \sigma \in \{\pm1\}$ lands on -2, 0 or $+2$; the outer two leave the alphabet, and 0 names a component with a radial

slot, which a tangential tensor does not have. What is left is pure Ω multiplication. Splitting $\nabla_1 T$ by whether the inherited slot is radial,

$$(\nabla_1 T)^{\sigma\alpha_1\ldots\alpha_q} = \Omega_l^{\mp N}\, T^{\alpha_1\ldots\alpha_q}, \qquad \text{all } \alpha_i \text{ tangential}, \tag{25}$$

$$(\nabla_1 T)^{\sigma\alpha_1\ldots 0_j\ldots\alpha_q} = -\, T^{\alpha_1\ldots\sigma_j\ldots\alpha_q}, \qquad \text{one slot radial}. \tag{26}$$

Both hold *exactly*, and are checked as such against `SurfaceGradient` applied to a rank-1 tensor with its radial component set to zero.

(26) is the extrinsic curvature: on a unit sphere the normal part of $\nabla_a v_b$ is exactly $-v$. (25) says

> **On a tangential tensor the intrinsic covariant derivative is $\eth$ applied component by component, up to $\sqrt{2}$, and it is closed.**

So the two operators differ by an algebraic term, not a differential one, and $\eth$ is not a poor relation of ∂^σ: it is the intrinsic derivative, exact on exactly the objects — tangential, spin-weighted — that it was invented for. Which of the two a caller wants depends on whether the radial directions are part of the problem.

7.5 What is implemented

`SurfaceGradient` is ∇_1: (20) and (22) without the r^{-1}, taking a rank-q tensor expansion to a rank-$(q+1)$ one.

No radial part. ∂^0 is $\partial/\partial r$, which an angular library cannot supply; but the *surface* gradient has no $\mathbf{e}_0$ component at all (D&T C.145), so the operator is complete without it and the radial derivative is the user's to provide. What that means for the result is that its $\sigma = 0$ block is stored and zero: a rank-$(q+1)$ tensor whose $\mathbf{e}_0$ slot vanishes is an ordinary tensor, and keeping it as one leaves the result composable with contraction, further gradients and the transform, for a third of the storage.

The result has no symmetry and keeps the operand's reality. ∇_1 of a symmetric tensor is not symmetric in the new slot against the old ones, and inferring a symmetry from an operator is the problem `Materialise` declined to solve. ∇_1 of a real tensor is real, so only the reduced set is computed.

It needed one thing phase 5 had left out. Every term is a coefficient of some component at fixed (l, m), and the shifted components need not be stored. `TensorExpansion::Coefficient` now answers for *any* representable component, resolving the stored case, a permutation relative, a reality relative by (16), a real block's reduced $m \geq 0$ storage, a pinning as imaginary, and a vanishing orbit. It is the spectral counterpart of phase 4's component accessor and closes the asymmetry noted above.

What the tests pin. On a scalar the operator is $\eth$ up to $\sqrt{2}$, which checks the Ω factors and nothing else. The metric trace of the *second* gradient of a scalar is the surface Laplacian, $-l(l+1)$ — and that runs through the connection terms, since ∂^- of the $+1$ component subtracts the 0 component, so the identity fails outright if the operator is $\eth$ applied component-wise. The chain rule (27) is checked at rank one, in the spatial domain. And the gradient of a real tensor is checked against the same field widened to a complex one, which exercises reading derived components and writing pinned ones.

Divergence and curl are contractions of the gradient (D&T C.6.2). Contraction is pointwise, so a caller evaluates the gradient and contracts with the machinery of §13; there is no separate operator.

One further property worth knowing. ∂^σ preserves the chain rule as an operator on fields, D&T (C.154)–(C.155): $\nabla(\mathbf{TP}) = (\nabla\mathbf{T})\mathbf{P} + \mathbf{T}(\nabla\mathbf{P})$ gives

$$\partial^\sigma(T^{\alpha\cdots}P^{\beta\cdots}) = (\partial^\sigma T^{\alpha\cdots})P^{\beta\cdots} + T^{\alpha\cdots}(\partial^\sigma P^{\beta\cdots}). \tag{27}$$

It does *not* hold coefficient by coefficient, because a product of fields is not a product of coefficients — which is the same fact that makes "the gradient of a product" an operation in two representations.

8 Quadrature

Colatitude. Gauss–Legendre with $n_\theta = l_{\max} + 1$ nodes, which integrates a polynomial of degree $2n_\theta - 1$ in $\cos\theta$ exactly. The nodes are the roots of P_{n_θ}, mapped by $\theta = \arccos(-x)$, and are symmetric about $\pi/2$.

Longitude. The trapezoid rule on n_ϕ equally spaced points, which is exact for $e^{i(m-m')\phi}$ only when $|m - m'| < n_\phi$. Analysing a band-$l_{\max}$ field needs $|m|, |m'| \le l_{\max}$, hence

$$n_\phi \ge 2l_{\max} + 1. \tag{28}$$

At $n_\phi = 2l_{\max}$ the orders $m = +l_{\max}$ and $m = -l_{\max}$ are the same discrete mode and cannot be separated. The library takes the smallest *fast* FFT length at or above the bound — restricted to $2^a3^b5^c7^d11^e13^f$ with $e + f \le 1$, the set FFTW has codelets for — because the naive $2l_{\max} + 2$ lands on $514 = 2 \times 257$ at $l_{\max} = 256$, and 257 is prime. The smooth choice there is 520, at about 1.5% more FFT work, and the FFT is not the expensive stage (§9).

Headroom. A product of two band-L fields has band $2L$, and $\int |f|^2$ for band-L f integrates a degree-$2L$ quantity. Both want a grid resolving more than the fields do. `Grid::ForBand(L, nMax, oversampling)` expresses that distinction — the band of the fields against the resolution of the grid — rather than leaving a caller to compute a degree and hope; the 3/2 dealiasing rule is oversampling = 1.5 and 2 is exact for a single product.

Part II
Numerical methods

9 The transform

9.1 The algorithm

The forward transform of a field at upper index n is

$$f^n_{lm} = \sum_i w_i \, X^n_{lm}(\theta_i) \, F_m(\theta_i), \qquad F_m(\theta) = \frac{2\pi}{n_\phi} \sum_j f(\theta, \phi_j) e^{-im\phi_j}, \tag{29}$$

so it is an FFT in longitude at each colatitude, followed by a weighted sum over colatitudes against the stored X. The inverse runs the same two stages in the other order. Both are separable exactly because (8) is a product of a function of θ and $e^{im\phi}$.

Cost. The FFT stage is $O(n_\theta\, n_\phi \log n_\phi)$ and the Legendre stage is $O(n_\theta\, l_{\max}^2)$, and at production sizes the second dominates completely. Measured at $l_{\max} = 256$, $n = 2$: a forward transform takes 19.0 ms, of which the FFT stage is **0.43 ms** and the Legendre stage **18.6 ms** — a factor of 44. Any optimisation effort that does not address the Legendre stage is misdirected, and this number is the reason the library's performance work concentrates entirely there.

Where the Legendre stage's time actually goes is less obvious than it looks. Single-threaded it runs at about 19% of the read bandwidth one core can achieve, so it is not DRAM-bound; what binds it is load/store throughput. Per stored value the inner loop does one 8-byte read that reaches memory and then, from cache, a read of the FFT bin and a read-modify-write of the coefficient — four memory operations, only one of which is a miss. Three hand-optimised rewrites of the kernel (weight hoisted, real and imaginary parts split, raw pointers) moved it by less than a factor of two in either direction.

9.2 Real fields

A real-valued field exists only at $n = 0$, where $f_{l,-m} = (-1)^m \overline{f_{lm}}$ by (16). The transform then uses a real-to-complex FFT and stores only $m \geq 0$, halving both the FFT work and the coefficient count. At $n \neq 0$ the same reduction is *invalid*: it assumes the self-relation $f^N_{l,-m} = (-1)^{m-N} \overline{f^N_{lm}}$, which holds only when f is its own conjugate, and a spin-weighted field is its own conjugate only at $N = 0$. The library therefore forbids real-valued fields at nonzero upper index in the type system, and the transform throws if asked.

9.3 Transforming is a projection

A component at upper index N has no content below degree $|N|$, because no harmonic there exists. So the forward transform of an *arbitrary* field discards whatever that field had at low degree, and evaluating the result does not reproduce the input. It reproduces its projection onto the span of the harmonics that exist.

This is not an error and not a loss of accuracy: it is what the expansion means. Once a field is band-limited — for instance because it came from an inverse transform — the round trip is an identity to rounding. Example 06 shows both, and prints 5×10^{-1} for the first and 4×10^{-14} for the second.

10 Computing the d-functions

This is where nearly all the arithmetic is, and where the library differs most from the reference texts.

10.1 Recursion in l, not in m

D&T describe two schemes, both at fixed degree: their (C.121) recurs in m at fixed N, and (C.122) recurs in N at fixed m. For stability they iterate (C.121) upward from $m = -l$ and downward from $m = +l$ and meet at $m = N\cos\theta$, seeding each end with the closed forms (11)–(12).

The library recurs in *degree*, at fixed (N, m). This is the scheme Woodhouse used, and the code is an evolution of his Fortran 77 implementation; D&T do not discuss it. The recursion is

$$d^l_{Nm} = f_1\, d^{l-1}_{Nm} - f_2\, d^{l-2}_{Nm}, \tag{30}$$

with

$$f_1 = \frac{(2l-1)\,[\,l(l-1)\cos\theta - Nm\,]}{(l-1)\sqrt{(l^2-N^2)(l^2-m^2)}}, \qquad f_2 = \frac{l\,\sqrt{((l-1)^2-N^2)\,((l-1)^2-m^2)}}{(l-1)\sqrt{(l^2-N^2)(l^2-m^2)}}. \tag{31}$$

Why this one. The transform consumes the values one degree at a time, in ascending order, for a fixed (N, θ) — see (29). A recursion in l produces them in exactly that order, so the same loop that fills the table can in principle feed the transform directly, and a truncated transform can stop early and pay for nothing it does not use. A recursion in m produces a whole degree at once and would have to be run to completion first.

What it costs. Of the coefficients in (31), only the $\cos\theta$ term depends on the colatitude; the four square roots are θ-independent and are read from a precomputed table of $\sqrt{k}$ and $1/\sqrt{k}$ — two vectors of $2l_{\max}+1$ entries, which is the whole of the auxiliary storage the recursion needs. The inner loop runs over m at fixed (l, N, θ) with unit stride on three rows, which vectorises.

The removable singularity. At $l = |N|+1$ the factor $1/(l-1)$ in (31) is $1/|N|$, which is a division by zero at $N = 0$; but $N/(l-1) = \operatorname{sign}(N)$ there, and d^{l-2} does not exist, so that degree is taken by a separate one-term branch with the singular factor cancelled analytically. The same one-term form appears at the "critical degree" $l = m_{\max}+1$, where the row stops growing.

10.2 The seed row and the boundary terms

The recursion needs starting values in two senses.

The seed row , at $l = |N|$, is a loop over m at a single degree. At $l = |N|$ the closed form reduces to

$$P^N_{l,-m}\big|_{l=|N|} = \sqrt{\binom{2l}{l+m}}\,\left(\sin\tfrac{\theta}{2}\right)^{l-m}\left(\cos\tfrac{\theta}{2}\right)^{l+m}, \tag{32}$$

and the binomial row is generated exactly by the standard step $\binom{2l}{k} = \binom{2l}{k-1}(2l-k+1)/k$. The row is at most $2|N|+1$ long — five entries for a rank-2 application — so this is not about cost; see §10.4.

The boundary terms are the values at $m = \pm l$, needed once per degree as the row grows. Evaluating (11) directly costs three $\log\Gamma$ and an exponential each time, which at $l_{\max} = 256$ is about 1.3×10^5 transcendental evaluations per upper index per pass. They obey a one-term recursion in l: writing $s = \sin(\theta/2)$, $c = \cos(\theta/2)$,

$$\frac{P^N_{l,-l}}{P^N_{l-1,-(l-1)}} = \sqrt{\frac{2l(2l-1)}{(l-N)(l+N)}}\;\; s\,c, \tag{33}$$

valid for $l > |N|$, seeded at $l = |N|$ where the square root is one and the value is $s^{2|N|}$ for $N \geq 0$ and $c^{2|N|}$ for $N < 0$. Both boundaries ride on the same recursion, because the ratio depends on N only through N^2 and because $P^N_{ll} = (-1)^{N+l}P^{-N}_{l,-l}$.

10.3 Why the recursion is the safer form

The closed form (11) is evaluated in logarithms, and it has to be: $\sqrt{(2l)!/\cdots}$ is of order 4^l and overflows long before the value it belongs to, which is bounded by one. The recursion (33) multiplies by a factor of order $2sc = \sin\theta \leq 1$ at each step, so nothing large is ever formed and no $\log\Gamma$ is needed. It is more robust than what it replaced, not less.

Measured against the closed forms over $l_{\max} = 128$, $|N| \leq 3$ and six colatitudes, the worst relative discrepancy is 2×10^{-13} in double and 9×10^{-17} in long double — very nearly the same multiple of ε, about 1000, in both, and growing linearly in $l_{\max}$ rather than with the number of entries compared. That says the drift belongs to the *closed form*, which exponentiates a

logarithm of size $O(l \log 4)$ and loses bits in proportion. The recursion's values are the better ones, and the test's tolerance is written as $20\, l_{\max}\varepsilon$ for that reason.

10.4 The poles, and a data race

At $\theta = 0$ and $\theta = \pi$ the ratio (33) is $0 \times \infty$. No special case is needed: the seed is already the exact 0 or 1 that the closed form's boundary branches return, and the factor is zero thereafter, so the recursion produces the right answer unaided.

The seed row (32) was moved off the closed form for a different reason. glibc's `lgamma` writes the global `signgam`, and the table is filled from every thread, so the closed form is a data race there — benign, since nothing reads `signgam`, but the only genuine one the library contains. Generating the binomial row instead removes the last $\log\Gamma$ from any threaded path, and the values are exact rather than merely accurate.

Two things follow that are worth recording. Grid construction is about 8% faster, which is what the arithmetic predicts: the boundary is two values per degree out of $2l + 1$, so however expensive a $\log\Gamma$ is, it is being paid on under 1% of the table. And ThreadSanitizer cannot confirm the absence of the race, because GCC's `libgomp` carries no TSan annotations and every OpenMP region is reported as racing with whatever ran before it; a Clang build against a `libomp` compiled with `LIBOMP_TSAN_SUPPORT` is the only way to get a real answer.

11 Storage and access

11.1 The table

Values are held as a flat array indexed by (N, i_θ) then by (l, m), contiguous in (l, m) for each (N, i_θ) — so a degree's row is a contiguous run, which is what the transform's inner loop wants. Offsets are closed form (`GSHIndices::OffsetForDegree`), not looked up.

The table is large. At $l_{\max} = 256$ with $n_{\max} = 2$ and all upper indices it is $5 \times 257 \times (257^2 - N^2)$ doubles, about 648 MB; at $l_{\max} = 512$ it is 5.4 GB. This is why the grid is a value-semantic handle over shared immutable state: copying one is a pointer copy, and `sizeof(Grid)` is 16 bytes. Before that change, twenty copies of a $l_{\max} = 128$ grid cost 1638 MB; afterwards, nothing.

11.2 The row supplier

Both transform loops take the row for a degree from an accessor rather than walking a single iterator across the whole block. The only thing either loop needs is *a contiguous run of values in (l, m) order for one (N, i_θ)*, and asking for it one degree at a time costs a few integer operations against a loop of length $(2l + 1)k$.

That seam exists so the values can come from somewhere other than a table. A second implementation runs the recursion into per-thread scratch inside the transform, and a grid can be asked for it at construction. It builds no table at all: construction at $l_{\max} = 256$ goes from 0.248 s and 648 MB to 0.015 s and nothing.

It is slower, and the measurement is worth keeping. On an 8-core laptop the generated path is 1.4–3.2× slower than the stored one in every configuration measured. Batching narrows the gap, since generation amortises over a chunk exactly as a table stream does, but *threads do not*: the ratio at eight threads is the ratio at one. The premise that the stored path is pinned at the DRAM roof while a generated one scales with cores is not visible on eight cores, and the deployment target has more memory bandwidth per core, not less. What survives is the memory saving, which is large, and a NUMA argument that only a two-socket machine can settle.

12 Batching, chunking and threading

12.1 The batch

The transform's primitive is "transform k fields sharing a grid, a degree and an upper index", with the single field as $k = 1$. What is being amortised is the Wigner block for that upper index, which is why fields at different upper indices cannot batch together — for a rank-2 tensor that means batching over radii within each N, not across components.

A batch is described by $(\text{count}, \text{stride}, \text{dist})$ rather than by contiguity: element j of field k lives at $j \cdot \text{stride} + k \cdot \text{dist}$. This is FFTW's advanced-interface form, and it costs nothing at the stage that actually reads the caller's layout, so it covers both arrangements the field layer produces:

	stride	dist
components stored component by component	1	FieldSize
components stored point by point	$n_{\text{components}}$	1

The descriptor's uniformity is a constraint on tensor storage , which was not obvious in advance. Its members must be evenly spaced, and in flat multi-index order the stored components sharing one upper index are scattered at no fixed spacing as soon as there is any symmetry. So a tensor's buffer is ordered *by upper index*, which makes each group a contiguous run on both the field side and the coefficient side. Ordered any other way, the batching the primitive exists for would be unreachable from the layer it was built for.

What batching is worth. About 2.3× at the optimum, not k. The gain saturates because the stage is bound by per-element load/store work rather than by table bandwidth, and batching removes $(k-1)/k$ of the *table* reads while leaving that untouched. It also *reverses* beyond an optimum: the coefficient array is k times larger, and once it stops fitting in cache it is streamed from DRAM too.

12.2 Chunking

Because of that reversal, a batched call processes a large batch in chunks of an internally chosen size rather than handing the caller's k straight to the inner loop. The chunk is

$$\text{chunk} \approx \frac{\text{cache}}{2 \times \text{copies} \times \text{bytes per coefficient block}}, \tag{34}$$

rounded to nearest, the factor of two being headroom for the streamed Wigner row and the FFT output.

"Copies" is not the thread count in both directions , and getting that wrong cost a factor of two. The forward transform gives every thread a private accumulator of chunk × coefficientSize, so its copies are its threads; the inverse gathers one block, shared and read-only, so it has one copy however many threads read it. Serving both with the thread count starved the inverse — at $l_{\max} = 256$, $k = 8$ on eight threads it returned a chunk of one where the whole batch fits, and the correction measured 2.0× on that direction.

One thing the rule still cannot see: on a multi-CCD or multi-socket machine a shared block is pulled into each cache domain that touches it, so the inverse's single copy is really one per domain.

12.3 Threading

Threading is an explicit per-call policy and defaults to sequential. The library creates threads because it was asked to and not because it can, and the rule it keeps is that **exactly one level threads**: an operation asked to run in parallel from inside an existing parallel region runs sequentially instead, so a caller parallelising over slices, components or realisations cannot nest with the transform, and neither can the table build.

The decomposition is over colatitudes, and the two directions differ. The inverse transform's colatitudes write disjoint rows and share only read-only input, so they divide with no reduction at all. The forward transform's colatitudes all contribute to every coefficient, so each thread accumulates into a private buffer and the partial sums are added at the end — partitioned rather than serialised: each thread sums every thread's partials for its own block of coefficients.

Measured , at $n = 2$, complex, per call:

	1 thread	2	4	8	16
$l_{\max} = 128$, forward	1.75 ms	1.89×	2.98×	**4.02**×	3.13×
$l_{\max} = 128$, inverse	1.77 ms	1.83×	3.07×	4.15×	**4.57**×
$l_{\max} = 256$, forward	12.16 ms	1.63×	2.51×	**2.68**×	2.23×
$l_{\max} = 256$, inverse	12.54 ms	1.69×	2.64×	**2.98**×	2.97×

Sixteen threads is not better than eight and is often worse: the machine has eight cores and sixteen hardware threads, and for memory-bound work the second thread on a core adds contention rather than throughput. Callers should ask for cores.

At $l_{\max} = 128$ on eight threads the stage reaches 39–40 GB/s against a measured machine roof near 42, so it is genuinely bandwidth-bound there and nothing further can be won without reducing the traffic itself.

13 What the type system enforces

The field algebra is built so that a statement which is not mathematically meaningful does not compile. What follows is the list, with the reason in each case.

13.1 The index algebra

Every node — an owning field, a view, or an expression — carries its upper index as a compile-time constant, and the operators enforce §2:

expression	upper index	requires
$f + g$, $f - g$	N	equal upper indices
fg	$N_f + N_g$	—
f/g	N_f	$N_g = 0$
$\overline{f}$	$-N_f$	—
$\lvert f\rvert$, $\lvert f\rvert^2$	0	—
$\operatorname{Re} f$, $\operatorname{Im} f$, $F(f)$	0	$N_f = 0$
$\int f \, \mathrm{d}\Omega$	—	$N_f = 0$

Three of these were wrong in the layer this replaced, and they are worth naming. **Conjugation reverses the upper index**: a quantity carrying $e^{-iN\psi}$ has a conjugate carrying $e^{+iN\psi}$. **Real and imaginary parts exist only at** $N = 0$, because for $N \neq 0$ the split is not preserved by a frame rotation and neither part is a component of anything. **Nonlinear functions likewise**, and integration with them, since $\int f \, \mathrm{d}\Omega$ vanishes identically unless $N = 0$.

The constraints are `requires`-clauses on the free operators, so an unlawful combination is an overload-resolution failure at the call site rather than an error inside a node — which also means the negative cases can be tested.

13.2 Real-valuedness

A node may be real-valued *only* at $N = 0$. Real-valuedness is not preserved by the frame rotation, so it is not a property any component of any tensor can have elsewhere; a library able to represent one could represent something that does not exist. The constraint is closed under every operation in the algebra — Re, Im, $|f|$, $|f|^2$ and $F(f)$ all land at $N = 0$ precisely because that is the only place their results could be real — so it is checked once, on the concept, and never re-derived.

13.3 Laziness, and the aliasing theorem

Operators build expression nodes; nothing is evaluated until a value is read. Every node in the layer is **pointwise and index-preserving**: the value at (i_θ, i_ϕ) reads its operands only at (i_θ, i_ϕ). Hence

> $u =$ expr is safe when u occurs in expr, and needs no temporary.

That is a theorem about the node set rather than a property of any one operator, which is why the invariant is stated in the stronger "index-preserving" form: a re-indexing view — a longitude shift, a transpose of the grid — would be pointwise in the loose sense and would break it. Operations that are genuinely not pointwise (gradients, raising and lowering, radial derivatives) live in the spectral layer for this reason and not by accident.

Operand storage follows the value category: a terminal passed as an lvalue is held by reference, everything else by value. The residual hazard — an lvalue terminal destroyed while an expression referring to it is alive — is the same one Eigen has and is not removable in C++.

13.4 Which components a tensor stores

Both reductions of §5 are computed by one `constexpr` walk of the group generated by the permutation symmetry together with, for a real tensor, the negation. Reaching a component twice by different routes is the interesting case rather than a failure: equating the two routes produces a constraint.

the two routes differ by	the orbit is
a sign, same conjugation	identically zero
conjugation, signs equal	real
conjugation, signs opposite	purely imaginary

The first arises without reality at all — it is the vanishing diagonal of an antisymmetric tensor. The others are the self-paired components, which always sit at $N = 0$, and are therefore exactly where a real-valued field is admissible.

One consequence reaches the interface: a component accessor cannot return one type. A stored component is a view, a symmetric-permutation relative is the same view, an antisymmetric relative is that view negated, a reality relative is its conjugate at the reversed upper index, and a vanishing orbit has nothing to return. Traversal over all 3^p components is therefore a compile-time loop, not a runtime one.

13.5 The two domains are not symmetric

On the spatial side a derived component is pointwise — $T^{-\alpha} = (-1)^N \overline{T^\alpha}$ — so a view plus an expression node gives it. On the spectral side the corresponding relation is (16), $T^{-N}_{l,-m} = (-1)^m \overline{T^N_{lm}}$, which *reverses the order index*. That is not a view over anything, so a tensor expansion exposes only its stored components and deriving is done in the spatial domain. This is the one place where the mirror between field and expansion breaks.

14 Validation

What each oracle actually pins, since a test suite that all passes is uninformative about which facts are load-bearing.

oracle	what it pins
`CheckWignerConvention`	That the stored value is X^N_{lm} with the upper index first, against all nine $l = 1$ values of D&T (C.115). Discriminating: the four entries with $N - m$ odd distinguish d^l_{Nm} from d^l_{mN}.
`CheckLegendre`	The $N = 0$ row against `std::sph_legendre`, which fixes the orthonormal scaling and the Condon–Shortley phase.
`CheckAdditionTheorem`	(13) over all $\lvert N\rvert, \lvert N'\rvert \le 40$ at $l_{\max} = 40$, to 1000ε. The strongest single check on the recursion: boundary values feed the two-term recursion at every higher degree, so an error anywhere propagates into it.
`CheckWignerBoundary`	The boundary recursion and the binomial seed row against the closed forms they replaced, to $20 l_{\max}\varepsilon$ — a tolerance argued from where the drift actually is (§10).
`CheckCoeff2Coeff`	Coefficient-to-coefficient round trips.
`TestRealFieldSymmetry`	(15) at $n = 2$ through complex transforms, and the rejection of real transforms at nonzero upper index.
batched transform tests	A batch of k against k separate calls, at *exact* equality. Batching widens the inner loop without reordering any sum, so any difference at all would mean it had been restructured rather than widened.
generated-path tests	The generating supplier against the stored table, again at exact equality: same recursion, same order, same rounding, only the storage differs.
reality tests	(14) at every component of every tensor, and that the stored set costs 3^p reals per point.
$\eth$ tests	(18), both of them.

On measuring this library. The development machine's clock scales under load, and the noise floor between back-to-back runs of identical code is about 10%. Comparisons must interleave the two versions in one process; figures from different sessions are worth less still. Within-run ratios — the stage split, the batching curve — are unaffected, because their terms are measured moments apart.

15 The GEMM restructure

This is the one substantial piece of performance work the library has not done, and `core-plan.md` §10 names it as the lever aimed at what actually binds. What follows is how it would work and why it should help, in enough detail to be argued with.

15.1 The Legendre stage is a matrix product

Written as it is implemented, the forward Legendre stage loops over colatitudes and accumulates:

$$f^n_{lm} \mathrel{+}= w_i \, X^n_{lm}(\theta_i) \, F_m(\theta_i) \qquad \text{for each } i, l, m. \tag{35}$$

Reordered so that the sum over θ is innermost, and at fixed upper index n, it is one matrix product *per order*:

$$f^n_{lm} = \sum_i D^{(n,m)}_{li} \, b^{(m)}_i, \qquad D^{(n,m)}_{li} = X^n_{lm}(\theta_i), \qquad b^{(m)}_i = w_i F_m(\theta_i). \tag{36}$$

Over a batch of k fields the right-hand side becomes a matrix and this is a GEMM,

$$\underbrace{(n_L(m) \times n_\theta)}_{D^{(n,m)}} \times \underbrace{(n_\theta \times k)}_{\text{weighted FFT output}} \longrightarrow \underbrace{(n_L(m) \times k)}_{\text{coefficients}}, \tag{37}$$

with $n_L(m) = l_{\max} - \max(|n|, |m|) + 1$ degrees surviving at that order. The inverse transform is the same matrix applied the other way round, $(n_\theta \times n_L) \times (n_L \times k)$, so *one stored matrix serves both directions* through a transpose flag.

At $l_{\max} = 256$ there are $2l_{\max} + 1 = 513$ such products per upper index, of heights n_L from 257 down to 1, all with $n_\theta = 257$ columns.

15.2 Why it should be faster

Not for the reason P2 originally gave. The stage is not short of arithmetic intensity against DRAM; it is short of *arithmetic per memory operation* (§9). Count them.

In (35), one stored value X is real and one FFT bin and one accumulator are complex, so each visit costs, in doubles, one load of X, two of the bin, and a load and a store of the accumulator — seven accesses — for two fused multiply-adds, or four flops. That is about 0.6 flops per double touched, and no amount of rewriting the loop moves it, which is what the three hand-optimised variants of P1 found.

A GEMM microkernel holds a tile of the output in registers and streams the two operands past it. With a tile of m_r rows by n_r columns, each step of the K loop loads $m_r + n_r$ doubles and performs $m_r n_r$ fused multiply-adds:

$$\frac{\text{flops}}{\text{double touched}} = \frac{2 m_r n_r}{m_r + n_r} \;\approx\; 5.3 \quad \text{at } m_r = 4,\ n_r = 8, \tag{38}$$

and the accumulator never leaves the registers, so its traffic vanishes entirely rather than being counted. That is an order of magnitude better than the loop it replaces, on precisely the metric that binds.

The DRAM traffic is unchanged. The table is still read once per batch; nothing about the restructure reduces it. So the GEMM buys nothing at all in the regime where the stage is bandwidth-bound — which it is, unbatched, on eight threads at $l_{\max} = 128$, where it already reaches 39–40 GB/s against a roof near 42. It buys everything in the batched regime, where the same stage runs at about a fifth of the roof and under a tenth of compute peak. Since phases 2 to 5 always batch, that is the regime that matters, but the distinction should be kept in view: a benchmark run unbatched will show the restructure doing nothing, and be right.

15.3 What has to change

The Wigner layout, which is the whole of step G's first option. (37) needs $D^{(n,m)}$ contiguous in (l, θ) for fixed (n, m). The table is currently contiguous in (l, m) for fixed (n, θ), which is the transpose of what is wanted. Storing it as $[n][m][l][\theta]$ makes each matrix a contiguous row-major block with leading dimension n_θ, and the total size is unchanged — summing $n_L(m)\, n_\theta$ over m gives the same $66\,045 \times 257$ doubles per upper index. `Wigner`'s existing `Storage` tag orders the (n, θ) axes only; this is a finer and different axis order, and it is what `RowMajor`'s deferred fate ([C7]) was waiting for.

The FFT output has to land in m-major order , since (37) wants, for one m, a contiguous (θ, κ) block. That means all the FFTs run *before* any of the Legendre stage, rather than interleaved per colatitude as now — the intermediate is $n_\phi \times n_\theta \times k$ complex, which is just the field, 2.1 MB at $l_{\max} = 256$ and $k = 1$.

The transpose is nearly free, for the same reason it was free in tier 1 (P6): a `plan_many` with output stride equal to the number of transforms and output distance 1 writes the result directly in $[m][\theta][\kappa]$ order. *Nearly*, and this paragraph used to say *free* and to specify a single call over all $n_\theta k$ rows. Both were corrected by measurement at `core-plan.md` M2. The strided write costs four to eight times as much when the number of transforms times 16 is a power of two, because successive orders then collide in the same cache sets; and a *small* block of colatitudes per call beats one large one by 1.7–2× while wanting eighty times less workspace, since a strided write over a few hundred bytes stays inside a couple of cache lines and one over tens of kilobytes does not.

The GEMM is real, and that is a gift. D is real while the data are complex, so there is no standard complex-times-real BLAS call — but there does not need to be. A complex $(n_\theta \times k)$ block with κ fastest *is* a real $(n_\theta \times 2k)$ block, since `std::complex` stores its parts adjacently. So the whole stage is `dgemm` with $N = 2k$, and the doubling helps most exactly where k is small.

15.4 What it does to threading

The products at different m write disjoint outputs. So the natural decomposition is over orders, and it needs **no accumulator and no reduction in either direction** — which is why §10 says this restructure subsumes [C11]. The forward transform's thread-private accumulators, which are correct at eight threads and are predicted not to survive 64–128, simply stop existing.

Two cautions. The work per order is not constant: $n_L(m)$ falls linearly in $|m|$, so a static split over orders is roughly twice as unbalanced as it looks, and the schedule should divide work rather than count. And a thread's share of the table is a set of whole (n, m) blocks, which is a better NUMA story than the current colatitude split — first touch during the table build can be made to match the split exactly, which it currently cannot.

15.5 What to expect, honestly

The shape is unfavourable. A GEMM reaches a large fraction of peak when all three dimensions are large; here $M = n_L$ is large only for small $|m|$, $K = n_\theta$ is a few hundred, and $N = 2k$ is between 2 and 16. Skinny-N GEMMs are the case general BLAS kernels handle worst, and the 513 products per upper index are individually small enough that call overhead is not negligible. A batched BLAS interface, where one call takes the whole set, is the right shape and not universally available.

So the honest expectation is a factor of two or three in the batched regime, not the order of magnitude that (38) suggests in isolation — and the measurement is the point of doing it.

15.6 What it composes with, and what it fights

The reflection composes, and helps more here than it would now. D&T (C.118) gives $P_{lm}^N(-\mu) = (-1)^{l+N} P_{l,-m}^N(\mu)$, so the matrices at $+m$ and $-m$ are related by reversing the colatitude and a sign that alternates with l. Splitting the input into its even and odd parts about $\theta = \pi/2$ turns the pair of products at $\pm m$ into two products of half the height, halving both the stored table and the arithmetic. This is the standard trick in production spherical-harmonic libraries. It is also exactly the reduction that P4 warned would trade bandwidth for an irregular access pattern in the *current* loop; inside a GEMM the same reduction is a clean halving of K, so the objection does not carry over.

It fights on-the-fly generation, and not merely by ordering. The recursion runs up degrees at fixed (n, θ), producing values in exactly the order the *present* layout stores them and exactly the wrong order for $[n][m][l][\theta]$. It would be easy to read that as an inconvenience with a transpose as its price. It is not. The recursion's output for one (n, θ) spans *every order at once*, so isolating the single (n, m) block that a per-order product wants means either keeping all of it — which is the table, and not having one is the entire purpose of generation — or re-running the recursion once per order, $2l_{\max} + 1$ times over. There is no third way, so the two are not combinable rather than merely awkward to combine, and `core-plan.md` [C17] refuses the pair at grid construction. Since the generated path measured slower anyway (§11), nothing is lost.

It costs the exact-equality tests, and buys a better one. A GEMM sums in whatever order its kernel chooses, so a GEMM path cannot be bit-compared against the current one; the batched-against-unbatched tests, which presently demand exact equality precisely because tier 1 does *not* reorder any sum, become tolerance comparisons on that path. `core-plan.md` [C12] turns that trade around by keeping *both* kernels permanently: the loop path is then an oracle for the matrix path on identical inputs, which checks the layout, the indexing, the FFT ordering and the accumulation together, and which does not exist today. The two are not independent in the d values, since both read the same recursion — but that half is pinned separately, against the $l = 1$ table and against `std::sph_legendre`.

Which BLAS. An earlier draft of this paragraph said Eigen was already a dependency and could supply the kernel. It is not: Eigen left the tree when GaussQuad dropped it, and nothing in the library names it now. So there is no free GEMM to start from, and `core-plan.md` [C14] takes the decision rather than defaulting into one — an optional BLAS, found and not fetched, under `GSHTRANS_WITH_BLAS`, in the pattern the optional `Interpolation` dependency established. A build without it does not offer the matrix kernel, exactly as a build without `Interpolation` does not offer `SplineDerivative`. It would be this library's first non-header-only dependency, which is why it is an option and not a requirement.

16 What has not been done, and why

Recorded so that the absences read as decisions.

The symmetry reductions of §4 would cut the table fourfold. They are not implemented because in the regime that matters they would buy little: batched, with a chunk that fits, the Legendre stage runs at about a fifth of the memory roof and under a tenth of compute peak, so halving the traffic addresses neither binding constraint. Unbatched they would be worth roughly 2×; batching already took that.

Reduced-precision storage is in the same position, and for the same reason. It also needs a tighter round-trip oracle than exists before the experiment would mean anything.

A transform-major layout with a GEMM is the one restructure aimed at what actually binds — the per-element load/store count — and it subsumes the question of how the forward transform should decompose above about sixteen threads. It is the natural next piece of performance work, and §15 sets it out in full.

Polar truncation, skipping the (m, θ) pairs where d^l_{Nm} is negligible — it falls off like $\sin^{|m|}\theta$ — is the one option that makes the problem smaller rather than making the machine work harder, and it reduces storage and arithmetic together. It is not implemented and probably dominates both of the reductions above.

The two open conventions of §1 want an answer from the literature rather than from a test.